Master Backend Engineer Interview Questions

157+ curated questions with concise answers

Data structures • Databases • System design • APIs • Concurrency
Microservices • Security • DevOps • Behavioral

Table of Contents

1. Data Structures & Algorithms

Q1. Explain time and space complexity. Why does it matter for backend services?

Big-O expresses growth of runtime/memory with input size. Backend services process variable, often large, workloads; poor complexity causes latency spikes, timeouts, and cost blowups under load.

Q2. Difference between an array and a linked list. When would you pick each?

Arrays: contiguous memory, O(1) index, cache-friendly. Linked lists: O(1) insert/delete at known nodes, no realloc. Use arrays for lookup-heavy work; linked lists for frequent middle inserts.

Q3. What is a hash table? What causes collisions and how are they handled?

Key→bucket via hash function; avg O(1) ops. Collisions arise from pigeonhole/poor hash. Handled by chaining (lists per bucket) or open addressing (linear/quadratic probing, double hashing).

Q4. Compare BFS and DFS with backend use cases.

BFS uses a queue, finds shortest unweighted path (e.g., social graph friend-of-friend). DFS uses stack/recursion, good for topological sort, cycle detection, dependency resolution.

Q5. What is a binary search tree vs a balanced tree (AVL/Red-Black)?

BST allows O(log n) search when balanced but degrades to O(n). AVL/RB self-balance on insert/delete guaranteeing O(log n). Most language std maps use RB trees.

Q6. Explain heaps and where they show up in backend systems.

Complete binary tree with heap order; O(log n) push/pop, O(1) peek. Used for priority queues, top-K queries, event schedulers, Dijkstra.

Q7. What is a trie and when is it preferable over a hash map?

Prefix tree of characters. Preferable for prefix search, autocomplete, IP routing tables, dictionary problems—shares prefixes to save memory.

Q8. Describe dynamic programming and give a backend-relevant example.

Solve overlapping subproblems by memoization/tabulation. Examples: edit-distance for fuzzy search, coin-change for billing, DP over intervals for rate-limiter windows.

Q9. What is the difference between recursion and iteration? Trade-offs?

Recursion uses call stack (clarity, stack overflow risk). Iteration uses loops (efficient, harder for tree/graph). Tail-call optimization is not guaranteed in JVM/Python.

Q10. Explain sorting algorithms: quicksort vs mergesort vs heapsort.

Quicksort: avg O(n log n), in-place, poor worst case. Mergesort: stable O(n log n), extra memory, good for linked lists/external sort. Heapsort: O(n log n) in-place, not stable.

Q11. What is amortized analysis? Give an example.

Average cost per op over a sequence. Dynamic array push: occasional O(n) resize amortizes to O(1) because doublings are rare.

Q12. What is a bloom filter and when would you use one?

Probabilistic set with false positives, no false negatives. Used in caches (Cassandra, CDN) to skip disk lookups for likely-absent keys.

Q13. Explain the sliding window technique with a real backend use case.

Maintain a window over stream/array in O(n). Used in rate limiting (requests per minute), moving averages for metrics.

Q14. What is the LRU cache algorithm and how would you implement it?

Least Recently Used eviction. Implement with hash map + doubly-linked list for O(1) get/put. Java LinkedHashMap or Guava CacheBuilder provide this.

Q15. Explain graph representations: adjacency list vs matrix.

List: O(V+E) space, fast for sparse graphs and neighbor iteration. Matrix: O(V^2), fast edge existence check, better for dense graphs.

Q16. What is topological sort and where is it applied?

Linear ordering of DAG nodes respecting edges. Used for build systems, task scheduling, dependency injection, database migrations.

2. Databases: SQL, Indexing, Transactions

Q17. Explain ACID properties.

Atomicity (all-or-nothing), Consistency (valid state transitions), Isolation (concurrent txns appear serial), Durability (committed data persists across failures).

Q18. What are isolation levels and their anomalies?

Read Uncommitted (dirty reads), Read Committed (non-repeatable reads), Repeatable Read (phantom reads possible), Serializable (none). Trade throughput for correctness.

Q19. Difference between clustered and non-clustered indexes.

Clustered: table rows stored in index order (one per table). Non-clustered: separate structure with pointers to rows. PK is usually clustered in InnoDB/SQL Server.

Q20. How does a B-tree index work and why is it used?

Balanced multi-way tree with sorted keys; O(log n) search, range scans, and disk-friendly (few page reads). Standard for relational engines.

Q21. What is a covering index?

An index that includes all columns needed by a query, so the engine avoids a table lookup. Speeds read-heavy workloads.

Q22. Explain normalization vs denormalization.

Normalization removes redundancy (1NF–BCNF) for write integrity. Denormalization duplicates data for read speed—common in analytics, feeds, wide-column stores.

Q23. What are JOIN types? When would each be used?

INNER (match both), LEFT/RIGHT (all rows one side), FULL (all), CROSS (cartesian). Use LEFT for optional relations; INNER for required.

Q24. Explain query execution plans and how to read one.

EXPLAIN shows operators (seq scan, index scan, hash join, nested loop) with cost estimates. Look for full scans on large tables, bad row estimates, spills to disk.

Q25. What is a deadlock and how do databases handle it?

Two txns each hold a lock the other needs. DB detects cycle and aborts one (victim). Prevent with consistent lock order, shorter txns, appropriate isolation.

Q26. What is optimistic vs pessimistic concurrency control?

Optimistic: assume no conflict, verify with version/timestamp at commit (good for low contention). Pessimistic: lock rows during txn (high contention).

Q27. Explain sharding and partitioning strategies.

Range, hash, list, directory-based. Aim for even distribution and locality. Trade-offs: cross-shard joins, resharding cost, hot partitions.

Q28. Difference between OLTP and OLAP databases.

OLTP: many small transactions, low latency (PostgreSQL, MySQL). OLAP: analytical, columnar storage, batch aggregations (Snowflake, BigQuery, Redshift).

Q29. What are materialized views and when are they useful?

Precomputed query results stored physically. Refresh on schedule or on change. Useful for expensive aggregations reused often.

Q30. Explain foreign keys and cascading actions.

Enforce referential integrity. ON DELETE/UPDATE options: CASCADE, SET NULL, RESTRICT, NO ACTION. Choose based on business semantics.

Q31. What is a transaction log / WAL?

Sequential record of changes written before data pages. Enables crash recovery, replication, and point-in-time restore.

Q32. What is the N+1 query problem and how do you avoid it?

One query loads N parents, then N queries load children. Fix with JOIN, batched IN queries, or ORM eager loading (dataloader pattern).

3. NoSQL & Caching

Q33. Explain CAP theorem.

In a network partition, choose Consistency or Availability. CP: MongoDB (default), HBase. AP: Cassandra, DynamoDB. CA is not achievable under partitions.

Q34. What is eventual consistency?

Given no new updates, all replicas converge. Common in AP stores. Client sees stale reads temporarily; trade for availability and low latency.

Q35. Compare document, key-value, wide-column, and graph databases.

Document: JSON-like (Mongo). KV: opaque values (Redis, DynamoDB). Wide-column: sparse tables (Cassandra, HBase). Graph: nodes/edges (Neo4j) for relationship queries.

Q36. What are the main uses of Redis?

Cache, session store, pub/sub, rate limiting, leaderboards (sorted sets), distributed locks (Redlock), streams for lightweight queues.

Q37. Difference between Redis persistence RDB and AOF.

RDB: periodic snapshot, compact, faster restarts, can lose recent writes. AOF: appends every write, durable, larger, slower recovery. Can combine.

Q38. What is cache stampede and how do you prevent it?

When a hot key expires, many requests hit backend simultaneously. Use request coalescing, mutex/single-flight, early recompute, jittered TTL.

Q39. Cache invalidation strategies?

Write-through, write-back, write-around, TTL expiry, event-driven invalidation, cache-aside. Choose by consistency needs.

Q40. What is a consistent hashing ring and where is it used?

Maps keys and nodes to a ring; only K/N keys move on membership change. Used in Cassandra, DynamoDB, Memcached, CDNs.

Q41. Explain read repair and hinted handoff in Cassandra.

Read repair reconciles replicas on read. Hinted handoff temporarily stores writes for down nodes and replays when they return.

Q42. What is a distributed lock? Pitfalls?

Coordinates exclusive access across nodes. Pitfalls: clock skew, expired lock while holder still working, split brain. Prefer fencing tokens over Redlock alone.

Q43. Difference between memcached and Redis.

Memcached: multi-threaded, strings only, LRU, no persistence. Redis: single-threaded per shard, rich types, persistence, pub/sub, Lua.

Q44. How does DynamoDB scale?

Auto-partitions by hash key across nodes with SSD storage. Provisioned or on-demand capacity. Watch for hot partitions on skewed keys.

Q45. What is MongoDB's replica set?

Primary + secondaries with automatic failover via election. Reads default to primary; can allow secondary reads with consistency trade-offs.

Q46. Explain write amplification in LSM trees.

LSM writes go to memtable, flush to SSTables, then compact—each byte may be rewritten many times. Trade write cost for fast writes and range reads.

Q47. When would you NOT use NoSQL?

Complex ad-hoc joins, strong multi-row transactions, mature reporting tools, or when relational integrity is central. Start relational unless scale demands otherwise.

4. System Design & Scalability

Q48. Vertical vs horizontal scaling.

Vertical: bigger machine, simple, hardware ceiling. Horizontal: more machines, needs stateless design, load balancing, harder to operate but elastic.

Q49. How would you design a URL shortener?

Base62 IDs from sequence or hash, KV store (Redis/DynamoDB), 301/302 redirect, analytics via async event pipeline, TTL/expiry, custom aliases.

Q50. Design a rate limiter.

Token bucket or leaky bucket per key in Redis; sliding window log/counter for accuracy. Centralize decisions; return 429 with Retry-After.

Q51. Design a news feed / timeline service.

Fan-out on write for read-heavy (push), fan-out on read for high-fan-out celebrities (pull), hybrid. Cache timelines; use Kafka to distribute events.

Q52. Design a chat system.

WebSocket gateway, message service, per-conversation partitioning, storage in wide-column store, push notifications, presence via ephemeral Redis keys.

Q53. Design a file storage service (Dropbox-like).

Chunk files, content-addressable storage, dedupe, metadata DB, resumable uploads, CDN, sync via delta encoding, client-side cache.

Q54. How do load balancers work? L4 vs L7?

L4 balances by IP/port (fast, protocol-agnostic). L7 inspects HTTP (routing by path/header, TLS termination, WAF). Examples: NLB vs ALB.

Q55. Explain CDN and its benefits.

Geographically distributed edge caches for static assets and cacheable API responses. Reduces latency, origin load; supports TLS, DDoS mitigation.

Q56. What is a circuit breaker?

Wraps calls to a dependency; opens after threshold failures, short-circuits requests, then half-opens to test recovery (Hystrix, resilience4j).

Q57. How do you design for high availability?

Redundancy at every layer, multi-AZ, health checks, auto-scaling, statelessness, graceful degradation, chaos testing, tested runbooks.

Q58. What is idempotency and how do you implement it in APIs?

Same request repeated yields same effect. Use idempotency keys stored server-side to dedupe writes (Stripe pattern).

Q59. Design a notification service (email/SMS/push).

Producer → queue → workers per channel with retries and DLQ. Templates service, user prefs, throttling, provider fallbacks, audit log.

Q60. Design a search autocomplete.

Prefix trie or ES completion suggester, popularity ranking, personalization, debounce on client, sub-100ms SLO, warm caches.

Q61. What is back-pressure and how to handle it?

Downstream signals it cannot keep up. Bounded queues, reactive streams, drop-oldest, load-shedding, 429 responses.

Q62. What are the SLIs, SLOs, and SLAs?

SLI: measured metric (latency, error rate). SLO: internal target. SLA: external contract with consequences. Error budgets drive release velocity.

Q63. Design a distributed unique ID generator.

Snowflake: timestamp+worker+sequence. UUIDv7 for time-ordered. Trade sortability, size, and coordination cost.

5. APIs: REST, GraphQL, gRPC

Q64. What are REST principles?

Stateless, uniform interface, resource-oriented URLs, HTTP verbs, HATEOAS (rarely enforced), cacheable responses, layered system.

Q65. Idempotent HTTP methods?

GET, PUT, DELETE, HEAD, OPTIONS are idempotent. POST is not by default (use idempotency keys). PATCH depends on implementation.

Q66. Difference between PUT and PATCH.

PUT replaces the resource; PATCH applies a partial modification (JSON Patch or merge patch).

Q67. What status codes would you use for validation errors, auth failures, rate limits?

400 (bad request), 422 (unprocessable entity), 401 (unauthenticated), 403 (forbidden), 429 (too many requests).

Q68. REST vs GraphQL trade-offs.

REST: simple caching, tooling, over/under-fetching. GraphQL: client-driven queries, single endpoint, complex caching, N+1 risk, needs dataloader.

Q69. REST vs gRPC.

gRPC: HTTP/2, protobuf, streaming, strong typing, low latency—great for internal services. REST: human-friendly, browser-friendly, ubiquitous tooling.

Q70. What is HATEOAS?

Hypermedia As The Engine Of Application State—responses include links to next actions so clients navigate dynamically. Rarely adopted in practice.

Q71. How do you version an API?

URI (/v1), header (Accept: application/vnd.x.v2+json), query param. Prefer additive, non-breaking changes and deprecation policies.

Q72. What is pagination and which strategies exist?

Offset/limit (simple, slow on deep pages), keyset/cursor (stable, fast, ordered), page tokens. Prefer cursor for large feeds.

Q73. How do you secure REST APIs?

TLS, authN (OAuth2/JWT/mTLS), authZ (RBAC/ABAC), input validation, output encoding, rate limiting, CORS, WAF, audit logs.

Q74. What is CORS and how does it work?

Browser same-origin protection; server opts in via Access-Control-* headers. Preflight OPTIONS validates method/headers before real request.

Q75. Design a webhook system.

Signed payloads, retries with exp backoff and DLQ, at-least-once delivery, idempotent consumers, subscription mgmt, timeouts, replay tool.

Q76. What are content negotiation and Accept headers?

Client indicates preferred representation (JSON/XML/CSV) via Accept; server chooses best match and returns Content-Type.

Q77. Explain gRPC streaming modes.

Unary, server-streaming, client-streaming, bidirectional. Built on HTTP/2 multiplexed streams.

Q78. How do you document APIs?

OpenAPI/Swagger for REST, protobuf files for gRPC, GraphQL SDL introspection. Include examples, error codes, auth flows.

Q79. What is API gateway and why use one?

Central entry point handling routing, authN, rate limiting, transformations, observability, canary. Decouples clients from microservices.

6. Concurrency & Async Programming

Q80. Process vs thread vs coroutine.

Process: isolated memory, expensive context switch. Thread: shared memory, cheaper. Coroutine: cooperative, user-scheduled, very cheap—great for I/O bound work.

Q81. What is a race condition? How do you prevent it?

Result depends on scheduling of concurrent accesses to shared state. Prevent with locks, atomics, immutable data, message passing, or transactions.

Q82. Explain mutex vs semaphore vs read-write lock.

Mutex: exclusive ownership. Semaphore: counts permits (bounded concurrency). RW lock: many readers or one writer, useful for read-heavy state.

Q83. What is a deadlock? Four Coffman conditions?

Mutual exclusion, hold-and-wait, no preemption, circular wait. Break any to prevent (e.g., lock ordering, timeouts, try-lock).

Q84. Difference between concurrency and parallelism.

Concurrency: dealing with many tasks (structuring). Parallelism: executing simultaneously (hardware). Concurrency enables parallelism but is not the same.

Q85. Explain thread pools and why they're used.

Reuse worker threads to bound resource use and avoid create/destroy overhead. Configure sizes per workload (CPU vs I/O bound).

Q86. What is async/await under the hood?

Compiler transforms function into a state machine; awaits yield control to an event loop until the future resolves. Enables high I/O concurrency.

Q87. What is the difference between blocking and non-blocking I/O?

Blocking: thread waits for I/O. Non-blocking: syscall returns immediately, event loop notifies on readiness (epoll/kqueue/IOCP).

Q88. What is the actor model?

Isolated actors with private state communicating via async messages (Akka, Erlang). Avoids shared-memory concurrency issues.

Q89. What is the Java Memory Model / happens-before?

Rules about visibility and ordering across threads. volatile, synchronized, and final establish happens-before edges.

Q90. Optimistic vs pessimistic locking in application code.

Optimistic: version/CAS retries on conflict; low contention. Pessimistic: lock rows/objects up front; high contention or long ops.

Q91. What is a fork/join pool?

Work-stealing pool for divide-and-conquer parallel tasks; idle workers steal tasks from busy ones.

Q92. Explain futures/promises.

Placeholder for a value produced asynchronously; chain via then/await; compose with all/any/race.

Q93. What is thread-local storage? When useful, when dangerous?

Per-thread state (context, tracing). Dangerous in thread pools where threads are reused—must be cleared to avoid leaks.

Q94. How do you debug a concurrency bug?

Reproduce with stress and jitter, thread dumps, deterministic logs with IDs, race detectors, minimize shared state, add invariants.

7. Microservices & Messaging

Q95. Monolith vs microservices trade-offs.

Monolith: simpler dev/deploy, easier txns. Microservices: independent scaling/deploy, tech diversity, but network complexity, distributed data, observability overhead.

Q96. What is service discovery?

Mechanism for services to locate each other dynamically (Consul, Eureka, DNS, Kubernetes Services).

Q97. What is a service mesh?

Sidecar proxy layer (Istio, Linkerd) handling mTLS, retries, timeouts, traffic splitting, and telemetry off the application.

Q98. Explain the Saga pattern.

Distributed transaction via a sequence of local txns with compensating actions. Orchestrated (central) or choreographed (event-driven).

Q99. What is event sourcing?

Persist state as an append-only log of events; project read models from it. Enables audit, replay, temporal queries.

Q100. What is CQRS?

Command Query Responsibility Segregation—separate write and read models, often paired with event sourcing for scalable reads.

Q101. Kafka vs RabbitMQ.

Kafka: distributed log, high throughput, replay, ordered per partition. RabbitMQ: smart broker, flexible routing, per-message ack, better for classic task queues.

Q102. Explain Kafka partitions and consumer groups.

Topics split into partitions for parallelism. A consumer group divides partitions among members; each partition processed by one consumer for ordering.

Q103. What delivery guarantees do queues offer?

At-most-once, at-least-once, exactly-once (rare/expensive). Most systems provide at-least-once—design consumers idempotent.

Q104. How do you handle poison messages?

Retry with backoff; move to dead-letter queue after threshold; alert; provide replay tool after fix.

Q105. What is exactly-once semantics in Kafka?

Combines idempotent producers, transactions, and read-process-write with __consumer_offsets in the same txn. Only within Kafka boundaries.

Q106. Choreography vs orchestration.

Choreography: services react to events; loosely coupled but hard to trace. Orchestration: central conductor; easier to reason about, single point of change.

Q107. Backend for Frontend (BFF) pattern.

Per-client API layer that aggregates/tailors downstream services for a specific UI, decoupling client changes from services.

Q108. What is the Outbox pattern?

Write domain change and event to the same DB txn (outbox table), then relay to broker. Guarantees consistency between DB and messaging.

Q109. How do you evolve message schemas?

Use versioned schemas (Avro/Protobuf), a schema registry, and enforce backward/forward compatibility rules.

8. Security & Authentication

Q110. Authentication vs authorization.

AuthN: who you are (login). AuthZ: what you can do (permissions). Different concerns, both required.

Q111. How does OAuth 2.0 work?

Resource owner grants a client access via authorization server; client receives access token (and optional refresh) to call resource server. Flows: authcode+PKCE, client credentials, device code.

Q112. What is OpenID Connect?

Identity layer on OAuth2 adding ID tokens (JWT) with user claims—standardized login.

Q113. JWT vs session cookies.

JWT: stateless, self-contained, hard to revoke, size overhead. Sessions: server state, easy revoke, requires sticky/shared store. Choose based on scale and revocation needs.

Q114. How do you securely store passwords?

Slow adaptive hash (bcrypt, scrypt, Argon2id) with per-user salt and appropriate cost factor. Never plain, MD5, or SHA-256 alone.

Q115. Explain CSRF and how to prevent it.

Attacker tricks browser into sending authenticated request. Mitigate with SameSite cookies, CSRF tokens, double-submit, checking Origin/Referer.

Q116. Explain XSS and mitigations.

Injecting scripts into pages. Mitigate with output encoding, template auto-escaping, CSP, HttpOnly cookies, input validation.

Q117. What is SQL injection?

Untrusted input concatenated into SQL. Prevent with parameterized queries, prepared statements, ORMs, least-privilege DB users.

Q118. What are the OWASP Top 10 categories?

Broken access control, cryptographic failures, injection, insecure design, misconfig, vulnerable components, ID/auth failures, integrity failures, logging failures, SSRF.

Q119. Symmetric vs asymmetric encryption.

Symmetric: one shared key, fast (AES). Asymmetric: keypair, slower, enables key exchange and signatures (RSA, ECDSA, Ed25519).

Q120. How does TLS handshake work?

Client/server negotiate ciphers, verify cert, derive session keys via (EC)DHE, switch to symmetric AEAD. TLS 1.3 completes in 1-RTT.

Q121. What is mTLS?

Both peers present certificates. Common for internal service-to-service auth in zero-trust and service mesh setups.

Q122. Explain RBAC vs ABAC.

RBAC: permissions via roles. ABAC: policy evaluates attributes (user, resource, environment). ABAC scales for fine-grained rules.

Q123. How do you handle secrets in production?

Secret manager (Vault, AWS Secrets Manager), env injection at runtime, rotation, least privilege, no secrets in git, encrypt at rest.

Q124. What is SSRF and how to prevent it?

Server tricked into fetching attacker-chosen URLs (e.g., cloud metadata). Prevent with allowlists, block private IP ranges, disable redirects, use metadata IMDSv2.

Q125. Explain rate limiting for abuse prevention.

Per-IP/user/token, sliding window or token bucket, aggressive on auth endpoints, coupled with captchas and anomaly detection.

9. DevOps, Containers & Observability

Q126. What problem do containers solve?

Packaging app + deps into a portable, isolated unit that runs consistently across environments. Lighter than VMs (share kernel).

Q127. Docker image vs container.

Image: immutable filesystem + metadata (layers). Container: running instance with writable layer, PID/network namespaces, cgroups.

Q128. Best practices for Dockerfiles.

Small base (distroless/alpine), multi-stage builds, pin versions, non-root user, minimize layers, .dockerignore, cache dependencies before source.

Q129. Explain Kubernetes core objects.

Pod (one or more containers), Deployment (rolling updates), Service (stable network), Ingress (external HTTP), ConfigMap/Secret, StatefulSet, Job/CronJob.

Q130. What is a Kubernetes Service and its types?

Stable virtual IP + DNS for pods. Types: ClusterIP, NodePort, LoadBalancer, ExternalName. Headless service for direct pod DNS.

Q131. Liveness vs readiness probes.

Liveness: restart if unhealthy. Readiness: gate traffic until ready. Wrong config causes crash loops or dropped traffic.

Q132. What is a rolling deployment vs blue/green vs canary?

Rolling: incremental pod replacement. Blue/green: switch traffic between two envs. Canary: send small % to new version and gradually ramp.

Q133. Explain CI vs CD (Delivery vs Deployment).

CI: build+test on every commit. Continuous Delivery: always releasable, manual promote. Continuous Deployment: automatic to prod.

Q134. What is Infrastructure as Code?

Provision infra via versioned declarative code (Terraform, Pulumi, CloudFormation). Reviewable, repeatable, auditable.

Q135. Explain the three pillars of observability.

Metrics (aggregates, dashboards), logs (structured events), traces (request path across services). Correlate via IDs.

Q136. What is distributed tracing?

Propagate a trace/span context across service calls to reconstruct a request's timeline (OpenTelemetry, Jaeger, Zipkin).

Q137. RED and USE methods.

RED (services): Rate, Errors, Duration. USE (resources): Utilization, Saturation, Errors. Complementary lenses for alerting.

Q138. What is a runbook? Why maintain one?

Documented steps for known incidents/operations. Cuts MTTR, spreads knowledge, complements automation.

Q139. Explain feature flags.

Runtime toggles decoupling deploy from release; enable canaries, kill switches, experiments. Manage lifecycle to avoid flag debt.

Q140. What is chaos engineering?

Deliberately inject failures (latency, crashes, region loss) in prod-like envs to validate resilience assumptions.

Q141. How do you approach cost optimization in cloud?

Right-size, autoscale, spot instances, savings plans, tiered storage, kill idle envs, tag & attribute cost, cache to reduce egress.

10. Behavioral & Scenario-Based

Q142. Tell me about a production incident you led. What did you learn?

Structure: context, your role, actions (mitigation → RCA → follow-ups), outcome, lessons. Emphasize communication and post-mortem culture (blameless).

Q143. Describe a tough technical trade-off you made.

Frame options, criteria (latency, cost, complexity, deadline), decision, outcome. Show you weigh trade-offs, not chase perfection.

Q144. Time you disagreed with a teammate on a design.

Show respect, data-driven argument, willingness to run a spike/PoC, aligning on the customer/business goal, gracefully committing after decision.

Q145. How do you decide when to refactor vs ship?

Assess risk, blast radius, business urgency; prefer incremental refactor tied to the feature (boy-scout rule) or dedicated tech-debt slots.

Q146. A service is 10x slower today than yesterday. Walk through debugging.

Check dashboards (latency, errors, deploys, infra), recent releases, DB slow log, cache hit ratio, dependency latency, saturation. Bisect by layer.

Q147. How do you handle competing priorities from multiple stakeholders?

Clarify goals, quantify impact, align with a PM/lead, communicate trade-offs, timebox, revisit weekly.

Q148. How do you keep learning?

Reading (papers, engineering blogs), building side projects, code review, incident reviews, mentoring, conferences.

Q149. Describe onboarding a new engineer.

Buddy system, curated docs, small starter tasks, pairing, weekly 1:1s, milestone at 30/60/90 days.

Q150. A downstream API you depend on is unreliable. What do you do?

Timeouts, retries with jitter, circuit breaker, fallback/cached response, alerting, negotiate SLA, consider bulkheads.

Q151. How would you migrate a large table with zero downtime?

Dual-write, backfill in batches, verify parity, switch reads with a flag, decommission old path, monitor rollback plan.

Q152. Time you made a mistake in production. What happened?

Own it, describe detection, mitigation, RCA, and durable fix (tests, guardrails, process). Show growth mindset.

Q153. Describe a system you're most proud of.

Context, constraints, your design decisions, measurable impact, what you'd do differently.

Q154. How do you write good code reviews?

Focus on correctness, design, readability, tests; be specific and kind; distinguish nits vs blocking; propose alternatives; approve promptly.

Q155. How do you approach on-call?

Prep with dashboards & runbooks; acknowledge quickly; communicate status; mitigate first, RCA later; document; feed learnings back into system.

Q156. How do you mentor junior engineers?

Set clear goals, pair on hard problems, review with why, invite them to design discussions, celebrate progress, give timely feedback.

Q157. Where do you want to be in 3–5 years?

Show ambition aligned with role: deeper technical mastery, mentoring, driving cross-team architecture, or moving toward staff/EM as fits.